

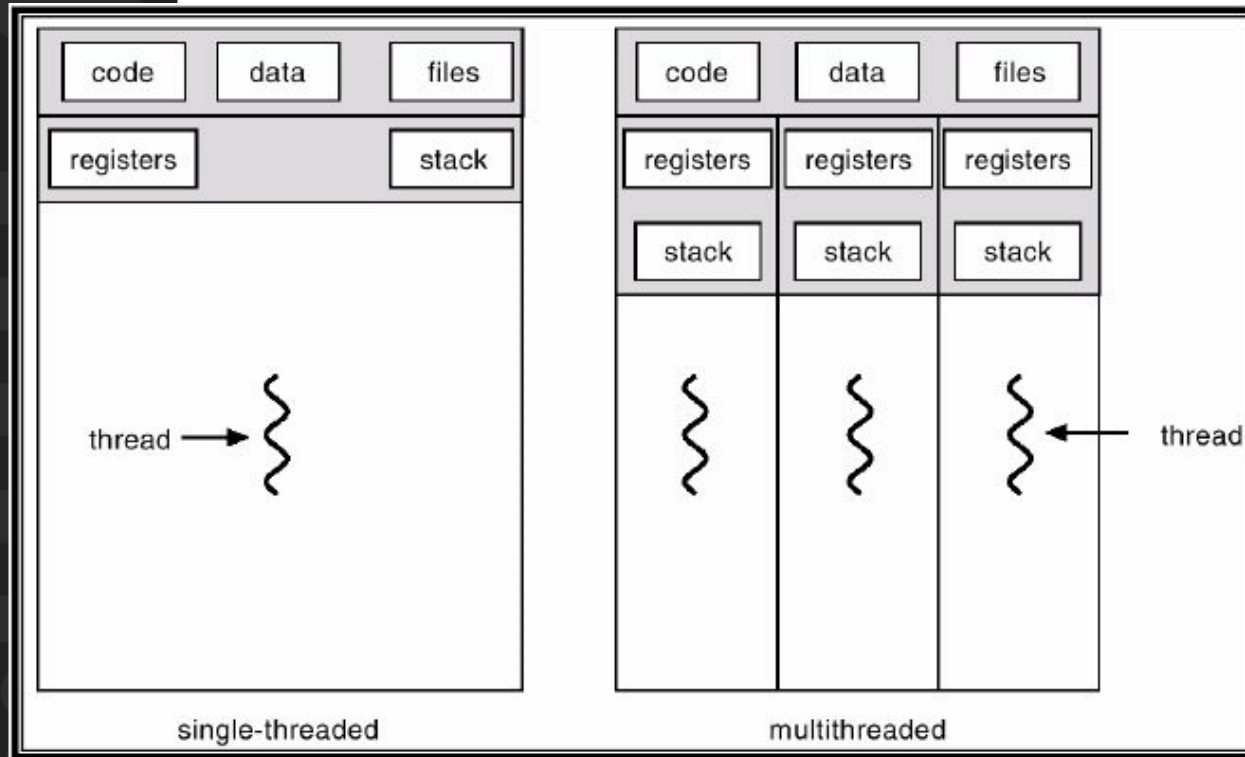


UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Centre de la Imatge i la Tecnologia Multimèdia

Lab Session 1: Threads



Definition



- In operating systems, threads are units of code that run concurrently to other tasks, sharing memory and resources.
- Multithreading vs Multitasking

... in Videogames

- Game Logic?
- AI?
- Networking?
- Art?
- ...

In Unity...

- Coroutines vs. Threads
 - Coroutine: simulate parallel execution by splitting work across frames
 - Threads: true parallel execution on different threads

When can this go wrong?

- Two threads modifying the same object
 - One thread reading while another modifies
 - Inconsistent frame data access
-
- Solution: Semaphores / Mutex / Locks

Threads in Unity

- – Using:

```
using System.Threading;  
Thread myThread = new Thread(myFunction)  
myThread.Start();
```

- myFunction must return void

- Docs:

<https://docs.unity3d.com/2020.1/Documentation/Manual/JobSystemMultithreading.html>

Ensure data consistency (Threads)

- Avoid simultaneous access (write/write or read/write).
 - You want to avoid changing a variable from 2 different threads at the same time
- Use locks to manage Access
 - You want to avoid changing a variable when it's being read by another thread(specially main)
- Example: Thread and Update handling the same data

```
public object myLock = new object();
```

```
Vector3 speed = new Vector3(0f,0f,0f);  
lock(myLock)  
{  
    pos=pos+speed;  
}
```

```
lock(myLock)  
{  
    transform.position = pos;  
}
```

Other considerations

- Unity doesn't like Threads
- Prefer Coroutines
- Be careful with Unity API calls
- Useful functions:
 - Thread.IsAlive()
 - Thread.Abort()
 - Thread.Join()

Coroutines in Unity

- Key: return IEnumerator
- yield return XXX allows frame-based delays
- XXX can be:
 - null: continue next frame
 - long task: wait till finished
 - other coroutines
- – Docs: <https://docs.unity3d.com/Manual/Coroutines.html>

```
IEnumerator coroutineInfiniteSecondPrinter()
```

```
while ((System.DateTime.UtcNow-myTime2).Seconds < 10f)  
{  
    yield return null;  
}
```

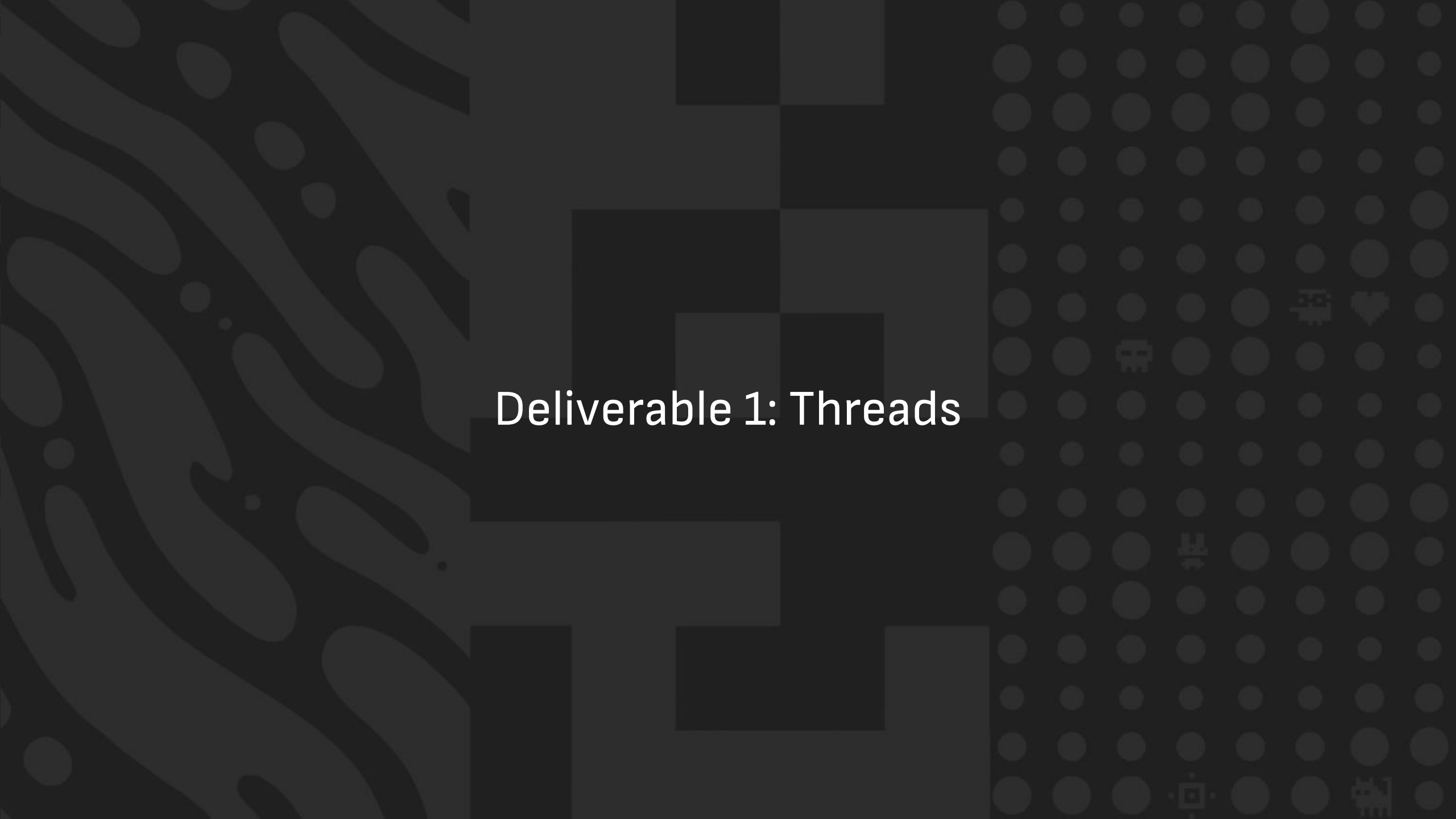
```
StartCoroutine("coroutineInfiniteSecondPrinter");
```

Other Coroutine-related functions

- Yield break
- StopCoroutine
- StopAllCoroutines

Some Documentation

- Unity Coroutines: <https://docs.unity3d.com/Manual/Coroutines.html>
- .NET Threads: <https://learn.microsoft.com/en-us/dotnet/api/system.threading.thread?view=net-6.0>
- Async Programming: <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/async/>
- Advanced: <https://www.youtube.com/watch?v=7eKi6NKri6I>

The background is a dark gray composition of three distinct patterns. On the left, there are large, flowing, organic shapes in a slightly lighter shade of gray. In the center, a grid of squares of varying shades of gray is visible. On the right, a dense pattern of small, light gray dots covers the area, with some larger, faint icons interspersed.

Deliverable 1: Threads

HANDOUT

- Practice threading
- Example: sorting algorithms
- Any long-running function can be used

To Do 0

- Download the P1 handout
- Available:
 - Array with random values
 - Empty list for GameObjects (for visual representation)
- Link in Atenea

```
float[] array;  
List<GameObject> mainObjects;  
public GameObject prefab;  
  
void Start()  
{  
    mainObjects = new List<GameObject>();  
    array = new float[30000];  
    for (int i = 0; i < 30000; i++)  
    {  
        array[i] = (float)Random.Range(0, 1000)/100;  
    }  
}
```

To Do 1

- Create a function to print arrays in console
 - Simple but useful for debugging

To Do 2

- Spawn geometries using array values
- Store GameObjects in a list to sort later

```
Instantiate(prefab, new Vector3((float)i / 1000,  
    this.gameObject.GetComponent<Transform>().position.y, 0), Quaternion.identity);
```


To Do 3

- Sorting the GameObjects list:
 - Complete function called `updateHeights()`.
 - Make each object's Y scale to match the value in the array
 - Track changes to avoid unnecessary calls

```
Vector3 currentScale = mainObjects[i].transform.localScale;  
mainObjects[i].transform.localScale = new Vector3(currentScale.x, array[i], currentScale.z);
```

To Do 4 & 5

- Single-threaded approach
 - Call all 3 functions to spawn and sort geometries
 - Sorting can be slow depending on count
- Multi-threaded approach
 - Call blocking function (BubbleSort) in a separate Thread

```
void bubbleSort()
{
    int i, j;
    int n = array.Length;
    bool swapped;
    for (i = 0; i < n- 1; i++)
    {
        swapped = false;
        for (j = 0; j < n - i - 1; j++)
        {
            if (array[j] > array[j + 1])
            {
                (array[j], array[j+1]) = (array[j+1], array[j]);
                swapped = true;
            }
        }
        if (swapped == false)
            break;
    }
}
```

To Do 6

- Call `updateHeights()` to update `GameObject` list
- Run it on Main Thread's update function to safely access Unity API

Class Exercise (Deliverable 1)

- Add another sorting algorithm (besides BubbleSort) and compare performance
 - Ensure frames continue running during sort
 - Finish in class to update grading
- Ref: <https://www.geeksforgeeks.org/sorting-algorithms/>